

Spark Structured Streaming:

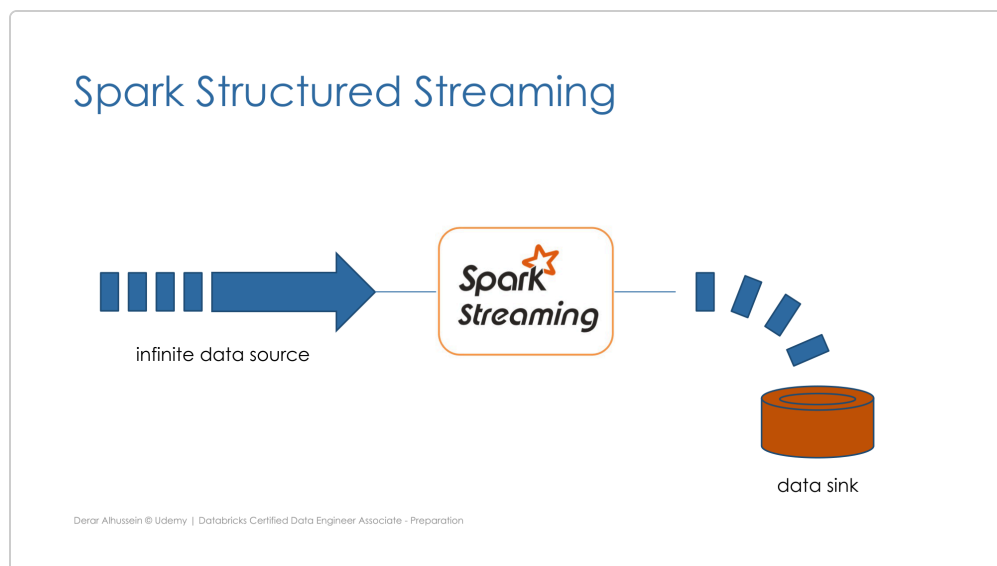
DataStreamReader und DataStreamWriter

Section 3 · Data Transformation and Modelling · Quelle: Udemy-Kursmaterial „Structured Streaming“, ergänzt mit Databricks-Dokumentation

Section 2 hat Auto Loader bereits im Kontext der Datei-Ingestion mit `spark.readStream` gezeigt, und Kapitel 1 dieser Section erwähnt Streaming Tables als Gold-Layer-Objekt. Dieses Kapitel liefert das fehlende Bindeglied dazwischen: die **generische Spark-Structured-Streaming-API** selbst – DataStreamReader und DataStreamWriter – unabhängig davon, ob die Quelle eine Datei, ein Delta-Table oder ein Message-Bus ist.

1. Was ist ein Datenstrom?

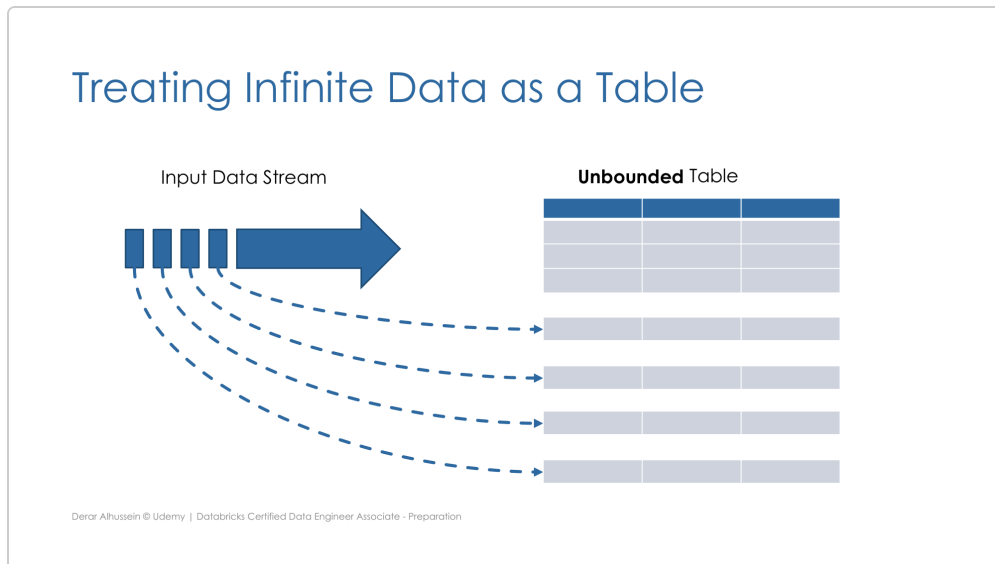
Ein **Datenstrom** ist jede Datenquelle, die kontinuierlich wächst – etwa neue JSON-Dateien, die in Cloud-Speicher landen, per CDC erfasste Datenbankänderungen, oder Ereignisse in einer Pub/Sub-Warteschlange wie Kafka. Um solche Quellen zu verarbeiten, gibt es grundsätzlich zwei Ansätze: den kompletten Datensatz bei jeder Aktualisierung neu zu verarbeiten, oder gezielt nur die seit dem letzten Lauf neu hinzugekommenen Daten zu erfassen. **Spark Structured Streaming** ist die Engine, die den zweiten, effizienteren Ansatz umsetzt.



Spark Structured Streaming liest kontinuierlich aus einer unendlichen Datenquelle und schreibt das Ergebnis inkrementell in eine dauerhafte Senke (Sink).

2. Das Konzept der unbounded Table

Der zentrale Kunstgriff von Structured Streaming: Ein kontinuierlich wachsender Datenstrom wird intern wie eine ganz normale, aber **unbegrenzte Tabelle** („unbounded table“) behandelt. Jede neue Dateneinheit im Stream entspricht schlicht einer neuen Zeile, die dieser Tabelle angehängt wird. Dadurch lassen sich auf einem Streaming-DataFrame nahezu dieselben Transformationen anwenden wie auf einem gewöhnlichen, statischen DataFrame.



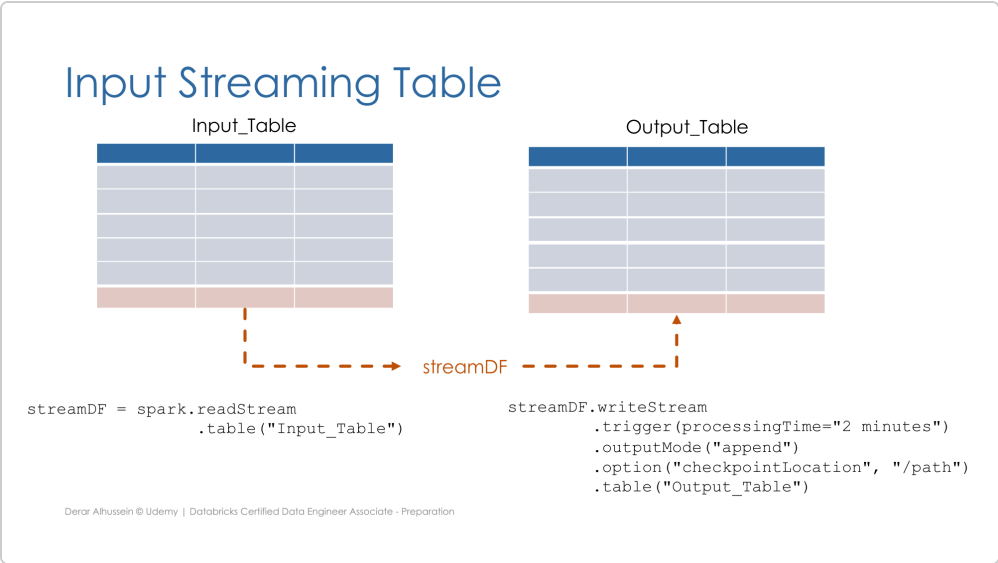
Jede neue Dateneinheit im Eingabe-Stream wird als neue Zeile in eine gedanklich unbegrenzte Tabelle eingefügt.

3. DataStreamReader und DataStreamWriter

Delta-Tabellen lassen sich direkt als Streaming-Quelle verwenden: `spark.readStream` liest dabei nicht nur die bereits vorhandenen, sondern auch alle künftig hinzukommenden Zeilen der Quelltable. Das Ergebnis ist ein Streaming-DataFrame, das sich mit `writeStream` in eine Zieltabelle schreiben lässt:

```
streamDF = spark.readStream.table("Input_Table")

streamDF.writeStream \
  .trigger(processingTime="2 minutes") \
  .outputMode("append") \
  .option("checkpointLocation", "/path") \
  .table("Output_Table")
```



spark.readStream liest kontinuierlich aus der Eingabetabelle; *writeStream* schreibt das transformierte Ergebnis in festgelegten Intervallen in die Ausgabetabelle.

4. Trigger-Intervalle: wann wird verarbeitet?

Die `trigger`-Methode legt fest, *wann* Structured Streaming nach neuen Daten sucht und diese verarbeitet:

Trigger	Aufruf	Verhalten
Nicht angegeben	–	Standard: alle 500 ms prüfen (Micro-Batch)
Fixes Intervall	<code>.trigger(processingTime="5 minutes")</code>	Verarbeitung in Micro-Batches im angegebenen Intervall
Getriggert Batch	<code>.trigger(availableNow=True)</code>	Verarbeitet alle verfügbaren Daten (in mehreren Micro-Batches) und stoppt danach von selbst

Beide getriggerten Varianten eignen sich für inkrementelle Batch-Verarbeitung (siehe Section 2, Kapitel 8): Der Stream läuft, verarbeitet alle aktuell vorhandenen neuen Daten und beendet sich danach automatisch, statt dauerhaft weiterzulaufen.

5. Output-Modi: wie wird geschrieben?

Modus	Aufruf	Verhalten
-------	--------	-----------

Append (Standard)	<code>.outputMode("append")</code>	Nur neu hinzugekommene Zeilen werden inkrementell an die Zieltabelle angehängt
Complete	<code>.outputMode("complete")</code>	Die Zieltabelle wird bei jedem Batch komplett neu berechnet und überschrieben

6. Checkpointing: der Fortschritt eines Streams

Über die Option `checkpointLocation` speichert Databricks den aktuellen Zustand eines Streaming-Jobs im Cloud-Speicher. Dieser Checkpoint erlaubt es der Engine, den Verarbeitungsfortschritt nachzuvollziehen – und nach einem Ausfall genau dort fortzusetzen, wo der Stream aufgehört hat. Wichtig für die Praxis: **Ein Checkpoint kann nicht zwischen mehreren Streams geteilt werden** – jeder einzelne Schreibvorgang (`writeStream`) benötigt einen eigenen, exklusiven Checkpoint-Pfad.

7. Garantien: Fehlertoleranz und Exactly-Once

Structured Streaming bietet zwei zentrale Garantien:

- **Fehlertoleranz** – dank Checkpointing in Kombination mit *Write-Ahead-Logs*, die den verarbeiteten Offset-Bereich jedes Trigger-Intervalls festhalten, kann der Stream nach einem Ausfall exakt dort fortsetzen, wo er unterbrochen wurde.
- **Exactly-Once-Verarbeitung** – da Streaming-Senken als *idempotent* konzipiert sind, führt ein mehrfaches Schreiben derselben, anhand des Offsets identifizierten Daten nicht zu Duplikaten im Ziel.

Beide Garantien setzen voraus, dass die Quelle **wiederholbar** ist (z. B. Cloud-Objektspeicher oder ein Pub/Sub-Dienst mit Offset-Verwaltung) – erst das Zusammenspiel aus wiederholbarer Quelle und idempotenter Senke ermöglicht echte Exactly-Once-Semantik über beliebige Fehlerfälle hinweg.

8. Nicht unterstützte Operationen

Auf einem Streaming-DataFrame lassen sich zwar die meisten Operationen genauso anwenden wie auf einem statischen DataFrame – einige Ausnahmen bestehen jedoch: **Sortierung** und **Deduplizierung** (siehe Section 3, Kapitel 13 zur Deduplizierung auf statischen DataFrames) sind auf einem unbegrenzten Datenstrom entweder zu aufwendig oder logisch nicht direkt möglich, da der vollständige Datensatz nie gleichzeitig vorliegt. Für solche Anwendungsfälle bietet Spark

fortgeschrittene Methoden wie **Windowing** und **Watermarking** an, die zeitlich begrenzte Ausschnitte des Streams betrachten.

Ergänzt aus der offiziellen Databricks-Dokumentation: `trigger(once=True)` gilt seit Databricks Runtime 11.3 LTS als **deprecated** – Databricks empfiehlt für sämtliche inkrementelle Batch-Workloads stattdessen durchgängig `trigger(availableNow=True)`, das inhaltlich denselben Zweck erfüllt (alle verfügbaren Daten verarbeiten, dann stoppen), dabei aber die Datenmenge in mehrere, besser steuerbare Micro-Batches aufteilen kann statt alles in einem einzigen Batch zu verarbeiten. Die Option `checkpointLocation` ist bei Verwendung von `trigger(availableNow=True)` zwingend erforderlich, damit jeder Lauf nachvollziehen kann, wo der vorherige Lauf aufgehört hat. Quelle: [Configure Structured Streaming trigger intervals – Databricks-Dokumentation](#)